pyspark.sql.DataFrame.withWatermark

`DataFrame.withWatermark(eventTime, delayThreshold)` [source]

Defines an event time watermark for this `DataFrame`. A watermark tracks a point in time before which we assume no more late data is going to arrive.

Spark will use this watermark for several purposes:

- To know when a given time window aggregation can be finalized and thus can be emitted when using output modes that do not allow updates.
- To minimize the amount of state that we need to keep for on-going aggregations.

The current watermark is computed by looking at the *MAX(eventTime)* seen across all of the partitions in the query minus a user specified *delayThreshold*. Due to the cost of coordinating this value across partitions, the actual watermark used is only guaranteed to be at least *delayThreshold* behind the actual event time. In some cases we may still process records that arrive more than *delayThreshold* late.

New in version 2.1.0.

Changed in version 3.5.0: Supports Spark Connect.

Parameters:

eventTime : *str*

the name of the column that contains the event time of the row.

delayThreshold : *str*

the minimum delay to wait to data to arrive late, relative to the latest record that has been processed in the form of an interval (e.g. “1 minute” or “5 hours”).

Returns:

`DataFrame`

Watermarked DataFrame

Skip to main content

Notes

This is a feature only for Structured Streaming.

This API is evolving.

Examples

```
>>> from pyspark.sql import Row
>>> from pyspark.sql.functions import timestamp_seconds
>>> df = spark.readStream.format("rate").load().selectExpr(
...     "value % 5 AS value", "timestamp")
>>> df.select("value", df.timestamp.alias("time")).withWatermark("time", '10
DataFrame[value: bigint, time: timestamp]
```

Group the data by window and value (0 - 4), and compute the count of each group.

```
>>> import time
>>> from pyspark.sql.functions import window
>>> query = (df
...     .withWatermark("timestamp", "10 minutes")
...     .groupBy(
...         window(df.timestamp, "10 minutes", "5 minutes"),
...         df.value)
...     ).count().writeStream.outputMode("complete").format("console").start
>>> time.sleep(3)
>>> query.stop()
```

Previous pyspark.sql.DataFrame.withMeta

Next pyspark.sql.DataFrame.write

Copyright @ 2026 The Apache Software Foundation, Licensed under the Apache License, Version 2.0.

Created using Sphinx 4.5.0.

Built with the PyData Sphinx Theme 0.13.3.